

Iterator

1. Explicación del patrón

Iterator es un patrón de comportamiento que permite recorrer secuencialmente los elementos de una colección sin exponer su estructura interna.

También se conoce como cursor.

La idea principal es separar dos responsabilidades:

- La colección se encarga de almacenar los elementos.
- El iterador se encarga de recorrerlos.

De esta forma, el cliente no necesita saber si la colección está implementada internamente como un array, una lista, una pila, un árbol u otra estructura. Solo necesita pedir un iterador y usar sus operaciones para avanzar por los elementos.

El iterador mantiene el estado del recorrido, por ejemplo la posición actual. Gracias a esto, pueden existir varios iteradores recorriendo la misma colección al mismo tiempo sin interferir entre ellos.

2. Cuándo se utiliza

Se utiliza el patrón Iterator cuando:

- Se quiere acceder al contenido de una colección sin exponer su representación interna.
- Una colección debe ofrecer una forma de navegar por sus datos sin revelar cómo están almacenados.
- Se necesita evitar un acoplamiento fuerte entre el cliente y la implementación concreta de la colección.
- Se quiere que los cambios en la implementación interna de la colección no afecten al código cliente.
- Se necesitan varios recorridos sobre una misma colección.
- Se quiere poder recorrer la colección de distintas maneras sin añadir una operación nueva a la colección por cada tipo de recorrido.
- Se deben permitir varios recorridos simultáneos sobre la misma colección.
- Se necesita una interfaz uniforme para recorrer distintos tipos de agregados o colecciones.
- Se quiere reducir la duplicación de código de recorrido en distintas partes de la aplicación.

3. Cómo aplicarlo

Para aplicar Iterator, se suele organizar el diseño en estos elementos:

Iterator

Define la interfaz común para recorrer una colección.

Operaciones habituales:

- `siguiente()` o `next()` : devuelve el siguiente elemento.
- `tieneSiguiente()` o `hasNext()` : indica si quedan elementos por recorrer.
- `anterior()` : opcional, permite retroceder en el recorrido.

Iterador concreto

Implementa el recorrido real sobre una colección concreta.

Suele guardar:

- Una referencia a la colección que recorre.
- La posición actual del recorrido.
- La lógica necesaria para avanzar, retroceder o comprobar si quedan elementos.

Agregado o colección

Define una interfaz para obtener un iterador.

Operación habitual:

- `crearIterador()` : devuelve un objeto iterador compatible con la colección.

Agregado concreto o colección concreta

Es la clase que almacena realmente los elementos.

Responsabilidades:

- Guardar los datos.
- Añadir elementos.
- Crear y devolver un iterador concreto cuando el cliente lo solicita.

Cliente

El cliente no recorre directamente la estructura interna de la colección.

En su lugar:

1. Pide un iterador a la colección.
2. Usa `tieneSiguiente()` para comprobar si quedan elementos.
3. Usa `siguiente()` para obtener cada elemento.
4. No necesita conocer cómo está implementada internamente la colección.

4. Forma práctica de aplicarlo

Pasos:

1. Detectar una colección cuyos datos se estén recorriendo desde fuera.
2. Evitar que el cliente acceda directamente a la estructura interna de esa colección.
3. Crear una interfaz `Iterador` .
4. Crear una interfaz `Agregado` o `Colección` con un método `crearIterador()` .

5. Implementar una clase `IteradorConcreto` que conozca cómo recorrer la colección.
6. Hacer que la colección concreta cree y devuelva su iterador.
7. Modificar el cliente para que recorra los elementos usando solo la interfaz del iterador.
8. Si se necesita otro tipo de recorrido, crear otro iterador concreto en lugar de modificar el cliente o sobrecargar la colección.

5. Idea clave

El recorrido de los elementos no debe estar mezclado con la lógica interna de la colección ni con el código cliente.

El patrón `Iterator` extrae esa lógica de recorrido y la coloca en un objeto independiente.

6. Ventajas prácticas

- Oculta la estructura interna de la colección.
- Reduce el acoplamiento entre cliente y colección.
- Permite cambiar la implementación interna de la colección sin modificar el cliente.
- Permite tener varios recorridos simultáneos.
- Permite crear distintas formas de recorrido mediante distintos iteradores.
- Evita llenar la colección con muchas operaciones específicas de recorrido.
- Facilita que distintas colecciones se recorran usando una interfaz común.

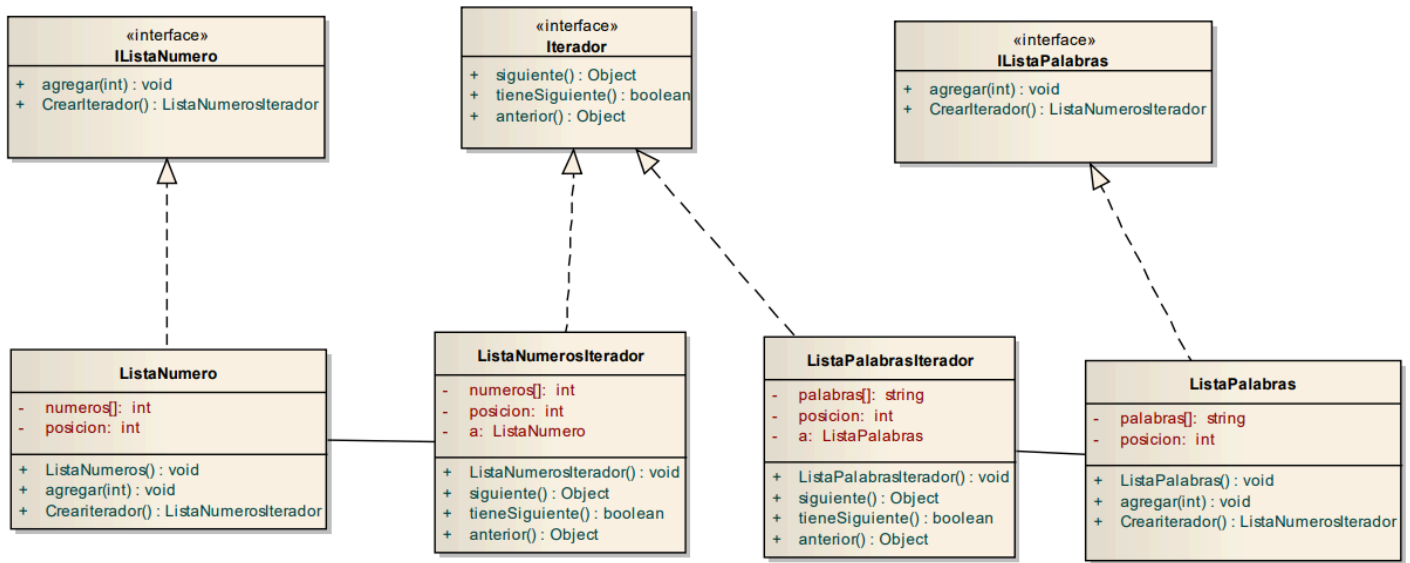
7. Cuidado

No siempre merece la pena aplicar `Iterator`.

Puede ser excesivo si la colección es muy simple y solo se necesita un recorrido básico. También puede ser menos eficiente que recorrer directamente algunas estructuras especializadas.

8. Plantilla

No se encontró "Pasted image 20260516180846.png".




Agregado

```

public interface Agregado {
    void agregar(int elemento);
    public Iterator CrearIterador();
    public Iterator CrearIterador2();
}
  
```



```
public class AgregadoConcreto implements Agregado{

    private int numeros[];
    private int posicion;

    public AgregadoConcreto() {
        numeros = new int[7];
        posicion=0;
    }

    @Override
    public void agregar(int elemento) {
        numeros[posicion++] = elemento;
    }

    @Override
    public Iterator CrearIterador() {
        return new IteratorConcreto(numeros);
    }

    @Override
    public Iterator CrearIterador2() {
        return new IteratorConcreto2(numeros);
    }
}
```



Interfaz ListaPalabras

```
//esta es equivalente a la de agregado
public interface I_ListaPalabras {
    public void agregar(String p);
    public ListaPalabrasIterator CrearIterador();
}
```

codesnap.dev



Iterator

```
//la implementan todo independientemente de como la recorra el agregado
public interface Iterator {
    public Object siguiente();
    public Object anterior();
    public boolean tieneSiguiente();
}
```

codesnap.dev



```
//hace como una cola
public class IteratorConcreto implements Iterator{

    private int numeros[];
    private int posicion;

    IteratorConcreto(int num[])
    {
        numeros=num;
        posicion=0; //porque vamos a recorrerlo como una cola
    }

    @Override
    public Object siguiente() {
        if(posicion<numeros.length)
            return numeros[posicion++];
        else
        {
            System.out.println("Ya no quedan numeros");
            return null;
        }
    }

    @Override
    public Object anterior() {
        if(posicion>0)
            return numeros[--posicion];
        else
        {
            System.out.println("No tiene anterior");
            return null;
        }
    }

    @Override
    public boolean tieneSiguiente() {
        if(posicion<numeros.length)
            return true;
        else
            return false;
    }
}
```

```
}
```

```
}
```



```
//hace como una pila
public class IteratorConcreto2 implements Iterator{

    private int numeros[];
    private int posicion;

    IteratorConcreto2(int num[])
    {
        numeros=num;
        posicion=num.length-1; //porque vamos a recorrerlo como una pila
    }

    @Override
    public Object siguiente() {
        if(posicion ≥ 0)
            return numeros[posicion--];
        else
        {
            System.out.println("No quedan elementos");
            return null;
        }
    }

    @Override
    public Object anterior() {
        if(posicion < numeros.length)
            return numeros[++posicion];
        else
        {
            System.out.println("No tiene anterior");
            return null;
        }
    }

    @Override
    public boolean tieneSiguiente() {
        if(posicion ≥ 0)
            return true;
        else
            return false;
    }

}
```



```
//equivalente a agregadoConcreto
public class ListaPalabras implements I_ListaPalabras{
    private String palabras[];
    private int posicion;

    ListaPalabras()
    {
        palabras = new String[7];
        posicion = 0;
    }

    @Override
    public void agregar(String p) {
        palabras[posicion++]=p;
    }

    @Override
    public ListaPalabrasIterator CrearIterador()
    {
        return new ListaPalabrasIterator(palabras);
    }
}
```



```
public class ListaPalabrasIterator implements Iterator{

    private String palabras[];
    private int posicion=0;

    ListaPalabrasIterator(String p[])
    {
        palabras=p;
    }

    @Override
    public Object siguiente() {
        if(posicion < palabras.length)
            return palabras[posicion++];
        else
        {
            System.out.println("No quedan palabras");
            return null;
        }
    }

    @Override
    public Object anterior() {
        if(posicion>0)
            return palabras[--posicion];
        else
        {
            System.out.println("No queda anterior");
            return null;
        }
    }

    @Override
    public boolean tieneSiguiente() {
        if(posicion<palabras.length)
            return true;
        else
            return false;
    }
}
```

}



```
public static void main(String[] args) {
    Agregado ln = new AgregadoConcreto();
    I_ListaPalabras lp = new ListaPalabras();

    System.out.println("Introduzco los numeros");
    ln.agregar(5);
    ln.agregar(1);
    ln.agregar(4);
    ln.agregar(68);
    ln.agregar(2);
    ln.agregar(0);
    ln.agregar(25);

    Iterator iterador = ln.CrearIterador();
    System.out.println("Vamos a sacar como una cola");
    while(iterador.tieneSiguiente())
        System.out.println(iterador.siguiente());

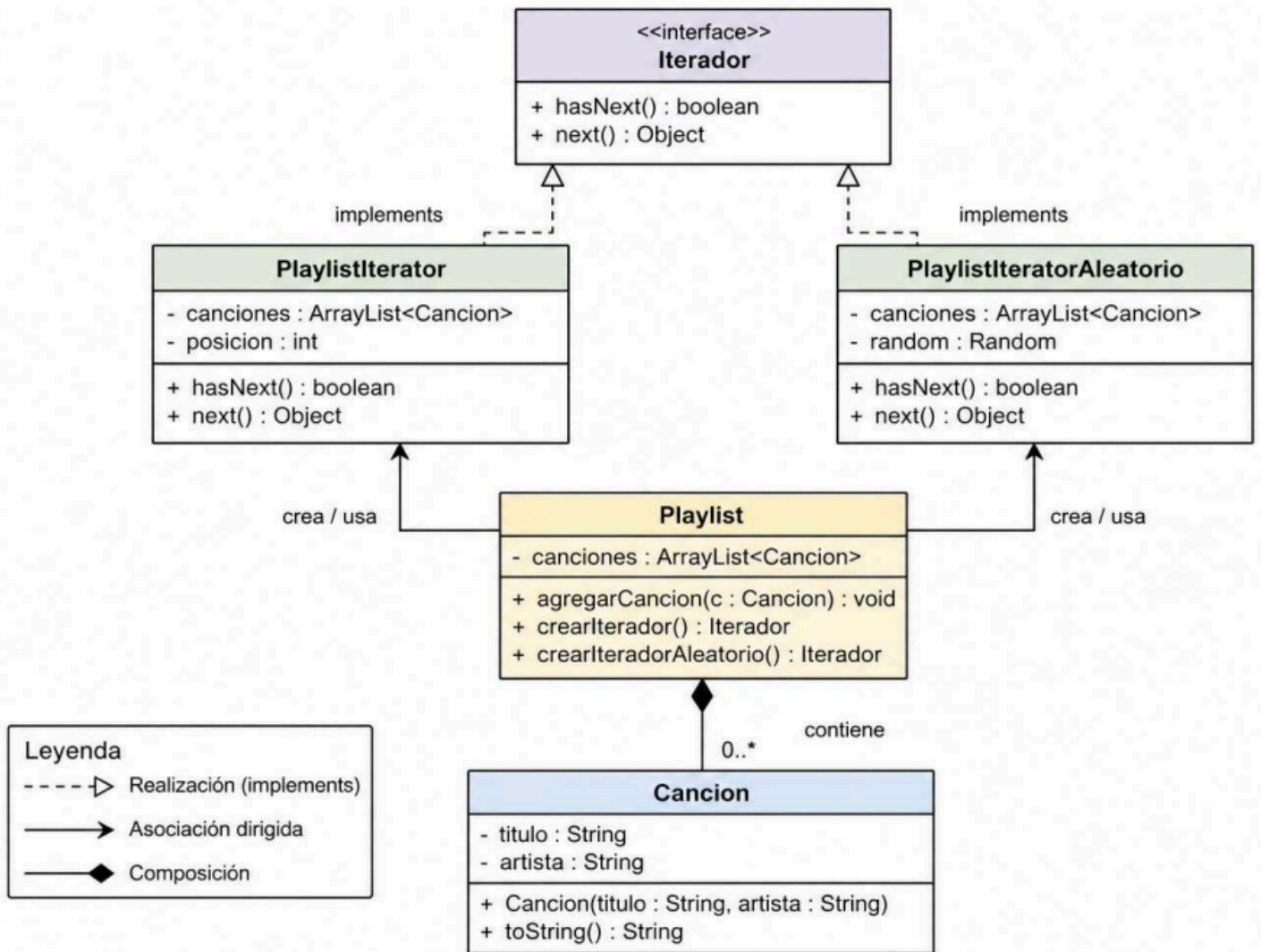
    Iterator iterador2 = ln.CrearIterador2();
    System.out.println("Vamos a sacar como una pila");
    while(iterador2.tieneSiguiente())
        System.out.println(iterador2.siguiente());

    lp.agregar("cinco");
    lp.agregar("uno");
    lp.agregar("cuatro");
    lp.agregar("sesenta y ocho");
    lp.agregar("dos");
    lp.agregar("cero");
    lp.agregar("venticuatro");

    Iterator iterador3 =lp.CrearIterador();
    System.out.println("Vamos a sacar las palabras como una cola");
    while(iterador3.tieneSiguiente())
        System.out.println(iterador3.siguiente());

}
```

9. Hueco para el ejemplo





Cancion

```
public class Cancion {  
    private String titulo;  
    private String artista;  
  
    public Cancion(String titulo, String artista) {  
        this.titulo = titulo;  
        this.artista = artista;  
    }  
  
    public String getCancion()  
    {  
        return titulo + " " + artista;  
    }  
  
}
```



```
public class PlaylistIteradorAleatorio implements Iterador{

    private ArrayList<Cancion> canciones = new ArrayList<>();
    private Random rand = new Random();
    int num;

    PlaylistIteradorAleatorio(ArrayList<Cancion> c)
    {
        canciones=c;
        num = rand.nextInt(c.size());
    }

    @Override
    public Object siguiente() {
        if(num<canciones.size())
            return canciones.get(num++);
        else
        {
            System.out.println("Ya no quedan numeros");
            return null;
        }
    }

    @Override
    public boolean tieneSiguiente() {
        if(num<canciones.size())
            return true;
        else
            return false;
    }

}
```

```
public class PlaylistIterador implements Iterador{
    private ArrayList<Cancion> canciones = new ArrayList<>();
    private int posicion;

    PlaylistIterador(ArrayList<Cancion> c)
    {
        canciones=c;
        posicion=0;
    }

    @Override
    public Object siguiente() {
        if(posicion<canciones.size())
            return canciones.get(posicion++);
        else
        {
            System.out.println("Ya no quedan numeros");
            return null;
        }
    }

    @Override
    public boolean tieneSiguiente() {
        if(posicion<canciones.size())
            return true;
        else
            return false;
    }
}
```



```
public class PlayList {
    private ArrayList<Cancion> canciones = new ArrayList<>();

    public void agregarCancion(Cancion c)
    {
        canciones.add(c);
    }

    public ArrayList<Cancion> getCanciones()
    {
        return canciones;
    }

    public Iterador crearIterador()
    {
        return new PlaylistIterador(canciones);
    }

    public Iterador crearIteradorAleatorio()
    {
        return new PlaylistIteradorAleatorio(canciones);
    }
}
```



I_Iterator

```
public interface Iterador {  
    public Object siguiente();  
    public boolean tieneSiguiente();  
}
```

codesnap.dev

```
public class EjemploIterator {

    public static void main(String[] args) {
        Playlist p = new Playlist();

        p.agregarCancion(new Cancion("Imagine ", "John Lennon"));
        p.agregarCancion(new Cancion("Bohemian Rhapsody ", "Queen"));

        Iterador it_a = p.crearIteradorAleatorio();
        Iterador it = p.crearIterador();

        System.out.println("Vamos a sacar aleatorio");
        while(it_a.tieneSiguiente())
        {
            Cancion c=(Cancion) it_a.siguiente();
            System.out.println(c.getCancion());
        }

        System.out.println("Vamos a sacar en orden");
        while(it.tieneSiguiente())
        {
            Cancion c=(Cancion) it.siguiente();
            System.out.println(c.getCancion());
        }

    }

}
```